

Dan Boneh 密码学第一季：第七课 认证加密

本笔记按照课程讲义的主线整理：主动攻击 → 认证加密定义 → 选择密文安全 → 加密与 MAC 的组合 → AEAD 标准 → TLS 与 WEP 案例 → CBC 填充预言机 → SSH 非原子解密攻击。

1. 本课要解决什么问题

前面的课程分别讨论了：

- 保密性：在选择明文攻击下实现语义安全，即 CPA 安全；
- 完整性：在选择消息攻击下实现 MAC 不可伪造性。

但是，仅有 CPA 安全的加密只能抵抗被动窃听，不能保证密文在传输过程中未被主动修改。本课的目标是同时实现：

1. **保密性**：攻击者无法得知明文内容；
2. **完整性与真实性**：攻击者无法构造新的、能够被接收方接受的密文。

这种同时提供保密性与密文完整性的方案称为：

认证加密，Authenticated Encryption，简称 AE。

学完本课，应当能够回答：

1. 为什么随机 IV 的 CBC 虽然 CPA 安全，却不能抵抗主动攻击？
2. 什么是密文完整性，为什么认证加密等于 CPA 安全加密再加密文完整性？
3. 为什么认证加密能够推出 CCA 安全？
4. Encrypt-then-MAC、MAC-then-Encrypt 和 Encrypt-and-MAC 有何区别？
5. 为什么 Encrypt-then-MAC 是最稳妥的通用组合？
6. AEAD 中的 Associated Data 是什么？
7. TLS 的 CBC 填充预言机如何逐字节恢复明文？
8. 为什么“先使用解密结果，再验证 MAC”会造成安全漏洞？

2. 为什么 CPA 安全还不够

2.1 CPA 安全主要面对被动窃听

CPA 安全保证：即使攻击者能请求任意明文的加密，也无法从挑战密文中区分两条等长明文。

但现实网络中的攻击者通常不只是观察密文，还能：

- 修改 IV；
- 翻转密文比特；
- 删除、插入或重排数据包；
- 把修改后的密文交给服务器；
- 观察服务器的响应、错误类型或响应时间。

这些能力属于主动攻击。CPA 安全本身并不承诺抵抗它们。

2.2 CBC 修改 IV 的例子

CBC 第一个明文分组的解密公式是：

$$m[0] = D_k(c[0]) \oplus IV.$$

假设其中包含：

```
dest = 80
```

攻击者希望把端口改成 25。令原字段为 *old*，新字段为 *new*，并修改：

$$IV' = IV \oplus old \oplus new.$$

接收方解密得到：

$$\begin{aligned} m'[0] &= D_k(c[0]) \oplus IV' \\ &= D_k(c[0]) \oplus IV \oplus old \oplus new \\ &= m[0] \oplus old \oplus new. \end{aligned}$$

因此，原字段 *old* 被替换成 *new*。攻击者不需要知道密钥，也不需要解密整个消息。

这说明：

CBC 的随机 IV 可以帮助实现 CPA 保密性，但不能阻止攻击者有控制地修改明文。

2.3 CTR 模式也具有可篡改性

CTR 模式可以抽象成：

$$c = m \oplus pad.$$

攻击者把密文改为：

$$c' = c \oplus \Delta,$$

解密结果就是：

$$m' = c' \oplus pad = m \oplus \Delta.$$

因此，流密码和 CTR 也允许攻击者控制明文中的比特翻转。讲义中的远程终端例子还利用 TCP 校验和与服务器 ACK 响应，把服务器变成判断明文猜测是否正确的预言机。

2.4 本节结论

根据消息需求，应选择：

- 只需要完整性和认证，不需要保密：使用安全 MAC；
- 同时需要完整性和保密：使用认证加密；
- 不能把仅有 CPA 安全的 CBC、CTR 等模式直接暴露给主动攻击者。

3. 认证加密的定义

3.1 接口

认证加密系统由加密算法 E 和解密算法 D 构成：

$$E : K \times M \times N \rightarrow C,$$

$$D : K \times C \times N \rightarrow M \cup \{\perp\}.$$

其中：

- K 是密钥空间；
- M 是消息空间；
- N 是 nonce 空间；
- C 是密文空间；
- $\perp$ 表示密文无效，解密方拒绝接受。

普通加密往往把任何格式正确的密文都映射为某个明文；认证加密则必须能够识别伪造或被篡改的密文并输出 $\perp$ 。

3.2 密文完整性

密文完整性通常记为 Ciphertext Integrity，简称 CI 或 INT-CTXT。

安全实验的基本过程是：

1. 挑战者随机选择密钥 k ；
2. 攻击者选择消息 $m_1, \dots, m_q$ ，并获得对应密文 $c_1, \dots, c_q$ ；
3. 攻击者输出一个密文 c ；
4. 若 c 是新的，并且 $D(k, c) \neq \perp$ ，攻击者获胜。

“新的”表示：

$$c \notin \{c_1, \dots, c_q\}.$$

攻击者优势为：

$$\text{Adv}_{CI}[A, E] = \Pr[D(k, c) \neq \perp \text{ 且 } c \notin \{c_1, \dots, c_q\}].$$

若所有高效攻击者的优势都可忽略，则该方案具有密文完整性。

3.3 认证加密

一个加密方案提供认证加密，当且仅当它同时满足：

1. 在选择明文攻击下语义安全，即 CPA 安全；
2. 具有密文完整性。

可以简写为：

$$\boxed{AE = CPA \text{ Security} + \text{Ciphertext Integrity}}.$$

3.4 为什么随机 IV 的 CBC 不是 AE

普通 CBC 解密会把每个合法长度的密文解密成某个比特串，并不会自动验证密文来源。攻击者修改 IV 或密文块后，解密算法通常仍然输出某条明文，而不是 $\perp$ 。

因此，攻击者很容易制造一个从未由加密接口产生、但能够正常解密的新密文，直接赢得密文完整性游戏。

4. 认证加密提供什么保证

4.1 消息真实性

若 Bob 收到密文 c ，并且：

$$D(k, c) \neq \perp,$$

则在没有密钥泄露的前提下，Bob 可以相信这个密文来自知道密钥的一方，而不是攻击者新构造的伪造密文。

但这里有一个重要限制：

认证加密本身不自动防止重放攻击。

攻击者可能重新发送以前见过的合法密文。因为密文本身是真的，认证验证仍会通过。

防重放通常需要额外加入：

- 序列号；
- 单调计数器；
- 会话标识；
- 已使用 nonce 的记录；
- 时间戳或协议状态。

4.2 不自动覆盖侧信道

认证加密的抽象定义通常把解密结果简化为“明文”或“ $\perp$ ”。现实实现可能泄露更多信息，例如：

- 填充错误与 MAC 错误返回不同提示；
- 两类错误的执行时间不同；
- 在验证前就根据明文长度读取网络数据；
- 内存访问、缓存、电磁或功耗泄露。

因此，数学上的 AE 安全不等于任意实现都安全。

5. 选择密文攻击与 CCA 安全

5.1 攻击者的能力

在选择密文攻击中，攻击者除了拥有加密接口，还拥有解密接口：

- CPA 查询：选择消息并获得加密结果；
- CCA 查询：选择密文并获得解密结果。

攻击者不能把挑战密文本身直接交给解密接口，否则实验毫无意义。但它可以提交与挑战密文非常接近的修改版本。

5.2 CCA 安全实验

攻击者提交两条等长消息：

$$m_{i,0}, m_{i,1}, \quad |m_{i,0}| = |m_{i,1}|.$$

挑战者选择秘密位 b ，并返回：

$$c_i \leftarrow E(k, m_{i,b}).$$

攻击者还可以查询其他密文的解密结果，但不能查询挑战密文本身。最后它猜测 b 。

若其区分优势：

$$\text{Adv}_{CCA}[A, E] = |\Pr[EXP(0) = 1] - \Pr[EXP(1) = 1]|$$

对所有高效攻击者都可忽略，则方案是 CCA 安全的。

5.3 为什么普通 CBC 不是 CCA 安全

假设挑战密文只有一个消息分组：

$$c = (IV, c[0]).$$

攻击者不能提交 c ，但可以提交：

$$c' = (IV \oplus 1, c[0]).$$

解密得到：

$$D(k, c') = m_b \oplus 1.$$

由此便能恢复 m_b ，进而判断挑战位 b 。攻击者没有查询原挑战密文，只查询了一个修改后的密文，所以这正是合法的 CCA 攻击。

5.4 认证加密推出 CCA 安全

讲义给出的核心定理是：

$$\boxed{AE \implies CCA \text{ Security}}.$$

直觉如下：

1. CCA 攻击者提交一个不是加密接口原样返回的新密文；
2. 由于密文完整性，这个新密文几乎必然被拒绝，解密结果为 $\perp$ ；
3. 因此，解密接口几乎不能向攻击者提供有用信息；
4. 剩下的攻击能力基本退化为 CPA 攻击；
5. CPA 安全保证攻击者仍无法区分挑战消息。

讲义中的定量界为：

$$\text{Adv}_{CCA}[A, E] \leq 2q \text{Adv}_{CI}[B_1, E] + \text{Adv}_{CPA}[B_2, E].$$

这里 q 是相关查询次数。该公式表达的是：如果存在有效 CCA 攻击者，就可以把它转化为密文完整性攻击者或 CPA 攻击者。

6. 如何组合加密与 MAC

设：

- $E(k_E, m)$ 是 CPA 安全的加密方案；
- $S(k_I, x)$ 是安全 MAC；
- 加密密钥 k_E 与认证密钥 k_I 应当独立，或通过安全 KDF 做域分离后派生。

历史上常见三种组合方式。

6.1 MAC-then-Encrypt：先 MAC，后整体加密

先计算明文的标签：

$$t = S(k_I, m),$$

再加密明文与标签：

$$c = E(k_E, m \parallel t).$$

流程：

$m \rightarrow \text{计算 MAC} \rightarrow m \parallel \text{tag} \rightarrow \text{整体加密} \rightarrow c$

历史上的 SSL/TLS CBC 记录协议采用了这一类顺序。

性质：

- 标签也被加密；
- 接收方必须先解密，才能检查标签；
- 若解密过程在 MAC 验证前泄露填充、长度或格式信息，容易形成预言机；
- 一般情况下，它不一定能抵抗 CCA 攻击；
- 讲义指出，在随机 CTR 或随机 CBC 等特定条件下可证明提供 AE，但实现必须避免额外泄露。

6.2 Encrypt-then-MAC：先加密，再认证密文

先加密：

$$c = E(k_E, m),$$

再对密文计算标签：

$$t = S(k_I, c).$$

发送：

$$c \parallel t.$$

流程：

$m \rightarrow \text{加密} \rightarrow c \rightarrow \text{对 } c \text{ 计算 MAC} \rightarrow c || \text{tag}$

接收方必须先验证标签:

1. 检查 $V(k_l, c, t)$;
2. 若失败, 直接返回 $\perp$, 不解密;
3. 只有标签正确时才执行 $D(k_E, c)$ 。

这是最重要的通用结论:

若加密方案 CPA 安全，MAC 满足课程所需的不可伪造性，则 Encrypt-then-MAC 总能提供认证加密。

它的优势是攻击者构造的无效密文在进入解密算法前就会被丢弃，因此更容易避免解密预言机。

IPsec 使用了这一类高层设计。

6.3 Encrypt-and-MAC: 分别加密明文和认证明文

分别计算：

$$\begin{aligned} c &= E(k_E, m), \\ t &= S(k_I, m), \end{aligned}$$

然后发送:

$$c \parallel t.$$

流程：

```

m → 加密 → c
└→ 对明文计算 MAC → tag
发送 c || tag

```

这种组合并不普遍安全，因为 MAC 的定义只保证不可伪造，并不保证标签隐藏明文信息。例如，一个仍不可伪造的 MAC 可能在标签中附带明文的某些比特，从而破坏保密性。

历史 SSH 协议使用过这一类结构，并且其分步解密处理还引出了本课后面的攻击。

6.4 三种组合对照

组合	发送内容	验证时机	通用安全结论	讲义中的协议例子
MAC-then-Encrypt	$E(m)$		$MAC(m)$	先解密后验证

组合	发送内容	验证时机	通用安全结论	讲义中的协议例子
Encrypt-then-MAC	c		MAC(c)	先验证后解密
Encrypt-and-MAC	$E(m)$		MAC(m)	通常需解密后验证

6.5 一个容易忽略的 MAC 条件

密文完整性要求攻击者不能把合法密文 (c, t) 改成另一个仍合法的新密文 (c, t') 。因此，即使消息 c 没变，攻击者也不能为它制造一个不同但有效的新标签。

这对应现代术语中的**强不可伪造性**。阅读不同教材时要注意：有些教材把它直接包含在“安全 MAC”的定义里，有些教材则区分普通不可伪造与强不可伪造。

7. AEAD：带关联数据的认证加密

7.1 为什么有些信息只认证、不加密

协议中常有一些字段必须让路由器或接收方在解密前读取，例如：

- 数据包类型；
- 协议版本；
- 长度；
- 序列号；
- 网络头部；
- 会话或用户上下文。

这些字段不需要保密，但必须防止篡改。因此出现了 AEAD：

Authenticated Encryption with Associated Data，带关联数据的认证加密。

7.2 AEAD 的输入

AEAD 加密通常接收：

$$(k, nonce, m, ad),$$

其中：

- m ：需要同时加密和认证的消息；
- ad ：Associated Data，只认证但不加密；
- $nonce$ ：同一密钥下按算法要求保持唯一或满足规定条件的公开值。

输出密文与标签：

$$(c, t) = AEAD.Enc(k, nonce, m, ad).$$

解密时必须提供完全相同的 $nonce$ 与 ad ，否则验证失败并返回 $\perp$ 。

7.3 常见标准

讲义列举：

- GCM: CTR 模式加密，再使用基于有限域运算的认证器；
- CCM: CBC-MAC 认证与 CTR 加密的组合；
- EAX: CTR 加密与 CMAC 认证的组合；
- OCB: 直接从 PRP 构造，每个分组大约只需一次分组密码运算，效率很高。

课程幻灯片中的性能数字来自当时的软件与硬件，只用于说明不同模式的效率差异，不代表今天实现的绝对性能。

7.4 nonce 重用

许多 AEAD 模式要求同一密钥下 nonce 绝不重复。以 GCM 为例，nonce 重用不仅可能泄露明文关系，还可能严重破坏认证安全。

因此，不能只保证 nonce“看起来随机”，还必须根据具体算法保证其唯一性或满足规范规定。

8. TLS 记录协议案例

讲义讨论的是 TLS 1.0/1.1/1.2 时代的 CBC 与 HMAC 设计，属于重要历史案例。现代 TLS 1.3 已取消这些旧式 CBC 密码套件，使用 AEAD。

8.1 状态与方向密钥

TLS 在两个通信方向上使用不同密钥：

$$k_{b \rightarrow s} \quad \text{和} \quad k_{s \rightarrow b}.$$

每个方向还维护独立的 64 位序列计数器。计数器从 0 开始，每处理一条记录就递增，用于把记录顺序纳入认证，并帮助抵抗重放与重排。

8.2 TLS CBC 记录加密

讲义中的高层步骤为：

1. 计算标签：

$$tag = S(k_{mac}, ++ctr \parallel header \parallel data).$$

2. 把 header、data 与 tag 填充到 AES 分组长度；
3. 使用 k_{enc} 和新的随机 IV 执行 CBC 加密；
4. 在记录前添加公开头部。

这是 MAC-then-Encrypt 类型的构造。

8.3 TLS CBC 记录解密

接收方：

1. 使用 k_{enc} 做 CBC 解密；
2. 检查填充格式；
3. 验证关于计数器、头部和数据的 MAC；
4. 任一步失败都拒绝记录。

在理想的抽象模型中，它可以提供认证加密。但如果第 2 步和第 3 步泄露不同信息，攻击者就获得了填充预言机。

8.4 旧版本 TLS 的两个问题

可预测 IV 与 BEAST

TLS 1.0 把当前记录的最后一个密文块作为下一条记录的 IV，即 chained IV。下一条记录的 IV 因此可被预测，破坏了 CBC 所需的 CPA 安全条件，并导致实际的 BEAST 攻击。

区分错误类型

旧实现可能：

- 填充错误时返回 `decryption_failed`；
- MAC 错误时返回 `bad_record_mac`。

攻击者由此知道修改后的密文是否产生了合法填充。

即使统一错误消息，若填充错误与 MAC 错误的处理时间不同，也可能通过统计响应时间恢复这一差别。

核心原则是：

解密失败时，不要向攻击者解释失败的具体原因，也不能通过时间等侧信道间接解释。

8.5 长度泄露

TLS 记录头部和网络流量会泄露记录长度。即使消息内容被正确加密，长度也可能揭示：

- 用户访问的页面；
- 医疗查询类型；
- 税务申报类别；
- 地图查询位置。

认证加密通常不隐藏密文长度。需要隐藏长度时，必须采用额外填充、流量整形或应用层设计。

9. WEP：完整性校验不等于密码学认证

WEP 使用流密码式加密，并在消息后附加 CRC：

$$(m \parallel CRC(m)) \oplus PRG(IV \parallel k).$$

CRC 适合检测随机传输错误，却不是安全 MAC。其线性性质为：

$$CRC(m \oplus p) = CRC(m) \oplus F(p).$$

攻击者想让明文发生差分 p 时，可以同时对待密文的消息部分异或 p ，并对 CRC 部分异或 $F(p)$ 。解密后消息被修改，但 CRC 仍然有效。

所以：

校验和只能检测偶然错误；面对能够主动选择修改方式的攻击者，必须使用带秘密密钥的密码学认证。

WEP 还存在 IV 重用、双次一密和相关 PRG 种子等其他问题。

10. CBC 填充预言机攻击

10.1 什么是填充预言机

假设服务器对攻击者提交的 CBC 密文解密，并泄露最后的填充是否有效：

$$Oracle(c) = \begin{cases} 1, & \text{填充合法,} \\ 0, & \text{填充非法.} \end{cases}$$

它不直接返回明文，却仍然提供了关于明文的一个比特信息。攻击者通过大量自适应查询，可以逐字节恢复明文。

10.2 CBC 中可控制的关系

对于相邻密文块 $c[0]$ 与 $c[1]$ ：

$$m[1] = D_k(c[1]) \oplus c[0].$$

攻击者不能控制 $D_k(c[1])$ ，但可以修改 $c[0]$ ，从而控制 $m[1]$ 中相应位置的异或变化。

10.3 恢复最后一个字节

设攻击者猜测 $m[1]$ 的最后一个字节为 g 。它把 $c[0]$ 的最后一个字节改为：

$$c'[0]_{last} = c[0]_{last} \oplus g \oplus 0x01.$$

解密后的最后一个字节为：

$$m'[1]_{last} = m[1]_{last} \oplus g \oplus 0x01.$$

如果猜测正确，即 $g=m[1]_{last}$ ，则：

$$m'[1]_{last} = 0x01,$$

形成合法的一字节填充。攻击者枚举 $g=0,\dots,255$ ，根据预言机响应即可确定该字节。

实践中还要排除原密文恰好具有其他合法填充等歧义，但不改变基本原理。

10.4 恢复倒数第二个字节

恢复最后一个字节后，攻击者先修改对应位置，让解密结果固定为 $0x02$ ；再枚举倒数第二个字节，使最后两个字节成为：

02 02

预言机返回合法时，就能确定倒数第二个明文字节。然后依次构造：

```
03 03 03
04 04 04 04
...
```

直到恢复整个分组。

10.5 为什么换密钥也不一定能阻止攻击

讲义中的 IMAP over TLS 案例说明：客户端会周期性发送结构固定的登录消息：

```
LOGIN "username" "password"
```

即使 TLS 在无效记录后重新协商新密钥，只要相同秘密在已知格式和位置中反复出现，攻击者仍可跨会话重复猜测，最终在数小时内恢复密码。

10.6 如何避免

Encrypt-then-MAC 可以直接阻断这类攻击：

1. 接收方先验证密文标签；
2. 攻击者修改的密文无法通过 MAC；
3. 无效密文不会进入 CBC 解密和填充检查；
4. 攻击者得不到填充是否合法的信息。

使用标准 AEAD 接口同样能够避免手工组合中的常见错误。

10.7 CTR 是否有 CBC 填充预言机

CTR 模式不要求消息按分组边界填充，因此没有 CBC/PKCS#7 意义上的填充预言机。但未经认证的 CTR 仍然可篡改，也可能因协议解析、格式检查或错误差异形成其他预言机。

正确结论不是“CTR 天生能够认证”，而是：

CTR 不需要分组填充，但仍必须与 MAC 正确组合或使用 AEAD。

11. 非原子解密：SSH 案例

11.1 什么叫原子解密

安全的认证解密接口应表现得像一个不可分割的操作：

$$AEAD.Dec(k, nonce, c, ad) = \begin{cases} m, & \text{认证成功,} \\ \perp, & \text{认证失败.} \end{cases}$$

在验证成功前，系统不应该：

- 把部分明文交给应用；

- 根据未认证明文分配资源；
- 根据未认证长度继续读取网络；
- 对不同内部错误产生可区分行为。

11.2 旧 SSH 数据包处理

讲义中的 SSH Binary Packet Protocol 使用 CBC，并对明文计算 MAC。解密步骤为：

1. 先只解密数据包长度字段；
2. 按该长度继续读取网络数据；
3. 解密剩余密文块；
4. 最后验证 MAC，并在失败时响应错误。

问题在于：长度字段在通过认证之前就已经被使用。

11.3 攻击如何泄露明文

攻击者拥有一个密文块：

$$c = AES_k(m),$$

并希望了解 m 。它把 c 放到数据包的首块位置，让服务器把解密结果的低 32 位解释为长度字段。

随后攻击者逐字节发送数据。当服务器已经收到该长度所指定的字节数时，就会继续处理并最终返回 MAC 错误。攻击者通过观察错误出现的时刻，得到解密块 m 的 32 个低位。

这里 MAC 本身没有被伪造，漏洞来自：

1. 解密不是原子操作；
2. 未认证的明文字段在验证前影响了外部行为。

11.4 设计改进

讲义给出的改进方向包括：

- 改成 Encrypt-then-MAC，先认证密文再解密；
- 长度字段可以公开，但必须纳入 MAC；
- 在长度字段后立即附加关于序列号与长度的认证标签；
- 调整包边界识别方式，避免使用未认证的解密结果。

现代设计的通用原则是：

在认证通过之前，不信任、不解析、不使用任何解密得到的数据。

12. 本课安全性质对照

安全性质	防御目标	攻击者可做什么	不自动防止什么
CPA 安全	被动窃听下的消息保密	查询任意明文的加密	密文篡改、CCA、重放、侧信道
MAC 不可伪造	消息完整性与来源认证	查询任意消息的标签	消息保密、重放

安全性质	防御目标	攻击者可做什么	不自动防止什么
密文完整性	不能构造新的有效密文	查询任意消息的加密	单独不定义消息保密
AE	CPA 保密性 + 密文完整性	主动修改和注入流量	重放、长度泄露、实现侧信道
CCA 安全	有解密接口时仍保持保密	查询非挑战密文的解密	完整性并非由定义单独表达
AEAD	AE + 认证公开关联数据	修改密文或关联数据	nonce 误用、重放、长度泄露

13. 常见误区

误区 1：密文看不懂，就无法被有意义地修改

错误。CBC 和 CTR 都具有可预测的比特翻转关系。攻击者不需要理解完整明文，也可能控制其中某些字段。

误区 2：CPA 安全就意味着能够抵抗主动攻击

错误。CPA 主要保证保密性，不保证密文完整性。主动攻击需要 AE 或 CCA 等更强安全目标。

误区 3：加密和 MAC 都安全，随便组合就安全

错误。顺序和认证对象非常重要。Encrypt-then-MAC 有通用安全结论，其他组合需要额外条件或可能直接不安全。

误区 4：接收方应先解密，确认格式后再验证 MAC

错误。解析或使用未认证明文会形成填充、格式、长度和时序预言机。Encrypt-then-MAC 应先验证再解密。

误区 5：CRC 可以代替 MAC

错误。CRC 没有秘密密钥且通常具有线性结构，攻击者可以同步修改消息和校验值。

误区 6：统一错误消息就一定消除了填充预言机

错误。不同错误路径仍可能有不同响应时间、连接行为或资源使用特征。

误区 7：认证加密自动防止重放

错误。旧的合法密文仍然可以通过认证。防重放必须依赖序列号、nonce 管理或协议状态。

误区 8：nonce 和密钥一样必须保密

错误。nonce 通常公开，但必须严格满足具体模式的唯一性或其他使用要求。

误区 9：AE 会隐藏消息长度

错误。大多数 AE/AEAD 方案仍会泄露明文长度或近似长度，需要额外填充和流量整形。

误区 10：MAC 最终会验证失败，所以提前使用一点明文没有关系

错误。攻击者可以从验证前的行为中获得信息。SSH 长度字段攻击正是这种错误的实例。

14. 必记结论与公式

认证加密定义：

$$AE = CPA\ Security + Ciphertext\ Integrity.$$

认证加密与选择密文安全：

$$AE \implies CCA\ Security.$$

CBC 第一分组的可篡改性：

$$m[0] = D_k(c[0]) \oplus IV.$$

修改 IV：

$$IV' = IV \oplus old \oplus new.$$

Encrypt-then-MAC：

$$c = E(k_E, m), \quad t = S(k_I, c).$$

MAC-then-Encrypt：

$$t = S(k_I, m), \quad c = E(k_E, m \parallel t).$$

Encrypt-and-MAC：

$$c = E(k_E, m), \quad t = S(k_I, m).$$

CBC 填充预言机的最后字节修改：

$$c'[0]_{last} = c[0]_{last} \oplus g \oplus 0x01.$$

15. 自测题

题目

1. 为什么随机 IV 的 CBC 可以 CPA 安全，却不具有密文完整性？
2. 认证加密由哪两个安全性质组成？
3. 密文完整性游戏中，攻击者怎样才算获胜？
4. 为什么 AE 能够推出 CCA 安全？
5. 三种“加密 + MAC”组合分别认证什么对象？
6. 为什么 Encrypt-then-MAC 可以在解密前拒绝伪造密文？
7. Encrypt-and-MAC 为什么可能泄露明文信息？
8. AEAD 中 Associated Data 的作用是什么？
9. 认证加密为什么不能自动阻止重放？
10. WEP 中的 CRC 为什么不能提供密码学完整性？
11. 填充预言机只返回一个比特，为什么仍能恢复整段明文？
12. 恢复 CBC 明文最后一个字节时，为什么要构造 0x01？
13. CTR 模式为什么没有 PKCS#7 填充预言机，却仍需要认证？
14. SSH 长度字段攻击的根本设计错误是什么？

15. 现代程序为什么应优先使用标准 AEAD 接口？

参考答案

1. 随机 IV 隐藏了相同明文的加密模式，但攻击者仍可修改 IV 或前一密文块，有控制地翻转解密明文；普通 CBC 不会认证这些修改。
2. CPA 安全和密文完整性。
3. 输出一个未由加密接口返回过的新密文，并且该密文解密后不为 $\perp$ 。
4. 密文完整性使攻击者提交的新密文几乎总被拒绝，解密接口基本失去作用；剩余能力退化为 CPA 攻击。
5. MAC-then-Encrypt 认证明文后整体加密；Encrypt-then-MAC 认证密文；Encrypt-and-MAC 分别加密明文并认证明文。
6. 标签覆盖密文本身。修改后的密文没有有效标签，所以接收方可先验证并直接丢弃，不调用解密算法。
7. MAC 安全只保证不可伪造，不保证标签隐藏消息；某个安全 MAC 的标签仍可能暴露明文的部分信息。
8. 它让协议头、序列号等公开字段不被加密，但仍受完整性保护。
9. 攻击者可以重新发送以前产生的合法密文，其认证标签仍然正确。需要序列号或状态额外检测。
10. CRC 无密钥且具有线性结构，攻击者能同时调整消息差分 and CRC 差分，使篡改后的数据继续通过检查。
11. 攻击者能够自适应修改前一密文块，并重复查询。每个“填充有效/无效”响应都排除猜测，逐字节累积后即可恢复整个分组。
12. PKCS#7 中一字节填充的合法形式就是 0x01。猜测正确时，修改后的最后一个字节会恰好变成 0x01。
13. CTR 不要求分组填充，但它仍具有比特可篡改性，也可能通过其他格式或协议响应泄露信息。
14. 系统在 MAC 验证前解密并使用长度字段，使未认证明文影响网络读取和错误响应。
15. 标准 AEAD 把加密、认证、关联数据和失败处理封装为统一接口，能减少组合顺序、验证顺序和错误处理方面的漏洞。

16. 一页复习提纲

1. CPA 安全只解决被动窃听下的保密性，不防止主动篡改。
2. CBC 可通过修改 IV 或前一密文块控制下一明文块的比特；CTR 也可被比特翻转。
3. 密文完整性要求攻击者不能制造新的、解密不为 $\perp$ 的密文。
4. 认证加密等于 CPA 安全加密加密文完整性。
5. AE 能够推出 CCA 安全，因为伪造的解密查询几乎都会被拒绝。
6. Encrypt-then-MAC 先加密，再认证密文；接收方先验证，再解密。
7. MAC-then-Encrypt 在一般情况下不保证安全，且容易因解密阶段的信息泄露形成预言机。
8. Encrypt-and-MAC 可能通过明文标签泄露消息信息。
9. GCM、CCM、EAX 和 OCB 是讲义列出的认证加密方案；AEAD 还能认证不加密的关联数据。
10. nonce 通常公开，但必须满足具体算法的唯一性或其他规定。
11. TLS CBC 的错误区分或时间差可形成填充预言机，逐字节恢复明文。
12. CRC 只能检测偶然错误，不能替代带密钥 MAC。
13. 认证通过前不能使用任何解密结果；SSH 长度字段攻击违反了这一原则。
14. AE 不自动隐藏长度，也不自动防止重放和侧信道。
15. 工程实践中优先使用成熟密码库的 AEAD 接口，不自行组合加密与 MAC。

17. 本课主线



这条主线体现了本课最重要的工程原则：**先认证，再相信数据；认证失败时，不向攻击者泄露内部处理细节。**